

LAB 10

Task 1:

CODE	EXPLANATION
<pre>[org 0x0100] start: mov ax, 0AAAAh mov bx, 0BBBBh mov cx, 0CCCCh push ax push bx push cx pop dx pop si pop di xchg dx, si mov ax, 0x4c00 int 0x21</pre>	➤ Take sample values in ax, bx and cx registers. ➤ Push ax register in stack ➤ Then bx. And finally cs ➤ Pop value of cx into dx. ➤ Pop value of bx into si ➤ Pop value of ax into di ➤ Exchange values of dx and si using xchg ➤ Terminate

Task 2:

Code	Explanation
<pre>org [0x0100] start: mov ax, 5555h mov bx, 2222h mov cx, 9999h call modifyRegisters mov ax, 0x4c00 int 21h modifyRegisters: push ax push bx push cx mov ax, 3333h</pre>	➤ ModifyRegisters saves the values of ax, bx and cx into stack. ➤ Then change values of ax and bx to 3333h and 1111h ➤ Perform subtraction of ax and bx ➤ Pop cx -> bx -> ax. ➤ Terminate program. ➤

mov bx, 1111h sub ax, bx pop cx pop bx pop ax ret	
--	--

Task 3:

Code	Explanation
org [0x0100] start: call sub1 mov ax, 0x4c00 int 21h sub1: push ax push dx call sub2 mov ah, 2 mov dl, 'A' pop dx pop ax ret sub2: push ax push dx call sub3 mov ah, 2 mov dl, 'B' pop dx pop ax ret sub3: push ax push dx mov ah, 2 mov dl, 'C' pop dx pop ax ret	➤ Program starts with start: label. ➤ It calls sub1. ➤ Sub1 push ax then dx into stack. ➤ It calls sub2. ➤ Sub2 push ax and dx into stack and it calls sub3. ➤ Sub3 push ax and dx into stack. ➤ Sub3 then store 2 into ah. ➤ And letter 'C' into dl ➤ Then it pop dx -> ax and return to sub2. ➤ Then in sub2, at store 2 into ah and 'B' into dl. ➤ Pops dx -> ax. And return to sub1. ➤ In sub1, it stors 2 into ah and 'A' into dl ➤ Pops dx -> ax and ret to start. ➤ Terminates the function

Task 4:

Code	Explanation
<pre> [org 0x0100] start: push word 6 push word 7 call multiply mov ax, 0x4c00 int 21h multiply: push bp mov bp, sp push bx mov ax, [bp+6] mov bx, [bp+4] mul bx pop bx pop bp ret 4 </pre>	• Starts and push sum value in stack. • Call multiply • Pushes bp and then stores value of sp into bp. • Pushes bx into stack. • Parameters: [bp+6] for 1st value, [bp+4] for 2nd value. • Mul bx multiplies ax by bx and stores result n ax. • Pop value in this flow bx -> bp. • Ret 4 pops return address and then increments sp by 4 to remove two parameter words.

Task 5:

Code	Explanation
<pre> [org 0x0100] Jmp start array: dw 1,2,3,4,5,6,7,8,9,10 start: mov ax, array push ax Mov cx, 10 mov ax, 10 push ax call sum_array mov ax, 0x4c00 int 21h sum_array: push bp mov bp, sp mov si, [bp+6] </pre>	• Mov address of 1st value of array into ax. • Pushes ax into stack. • Mov 10 into ax and pushes into ax then call the subroutine. • Pushes bp into stack. • Mov the value of sp into bp then mov 1st address into si and 2nd address into cx. • Clear the ax. • Add the values into ax register. • ret 4 clean the parameters

<pre> mov cx, [bp+4] xor ax, ax loop_sum: add ax, [si] add si, 2 dec cx jnz loop_sum pop bp ret 4 </pre>	
---	--

Task 6:

Code	Explanation
<pre> [org 0x0100] start: push word 0100h push word 00F0h call findMin push word 0005h push word 0010h call findMin mov ax, 0x4c00 int 21h findMin: push bp mov bp, sp mov ax, [bp+6] cmp ax, [bp+4] jle keep_ax mov ax, [bp+4] keep_ax: pop bp ➤ ret 4 </pre>	➤ It actually: ➤ Compares [bp +6] and [bp +4] and return smaller into ax. ➤ Ret 4 clear the parameters. ➤

Task 7:

Code	Explanation
<pre>[org 0x0100] start: push 0200h push 0100h call findMin_caller_cleanup add sp, 4 mov ax, 0x4c00 int 21h findMin_caller_cleanup: push bp mov bp, sp mov ax, [bp+6] cmp ax, [bp+4] jle fm_done mov ax, [bp+4] fm_done: pop bp ret</pre>	➤ Label findMin_caller_cleanup uses ret so that it return parameters on the stack. ➤ In the subroutine it: ➤ Stores values into ax and compare it with second value. ➤ Before pushes: SP was S0 and after two pushes it will be S0 - 4. ➤

Task 8:

Code	Explanation
<pre>[org 0x0100] array: dw 2,4,6,8,10 start: mov ax, array push ax mov ax, 5 push ax call avg_array mov ax, 0x4c00 int 21h avg_array: push bp mov bp, sp</pre>	• start pushes array address and count, then calls avg_array. • avg_array sets up BP, pushes the same two values, and calls sum_array. • sum_array adds all numbers, returns sum in AX, and cleans its own parameters (ret 4). • Back in avg_array, AX holds the sum; it divides by count (div bx) to get the average. • avg_array restores BP, does ret 4, and returns average in AX. • Example: [2,4,6,8,10] → sum = 30 → average = 6.

```
push word [bp+6]
push word [bp+4]
call sum_array
```

```
mov bx, [bp+4]
xor dx, dx
div bx
```

```
pop bp
ret 4
```

```
sum_array:
  push bp
  mov bp, sp
  mov si, [bp+6]
  mov cx, [bp+4]
  xor ax, ax
sum_loop:
  add ax, [si]
  add si, 2
  loop sum_loop
  pop bp
  ret 4
```